

Assignment Questions

Name: Chinmay N. Yalawatti

USN: 01FE24BEC258

Intermediate Level:

1) **Resize an Image Using Different Interpolation Methods**

Alters structural resolution dimensions using mathematical resampling strategies. Compares Nearest-Neighbor (fastest, blocks), Bilinear (smooth, 2x2 local average), and Bicubic (sharpest, 4x4 cubic spline fitting) interpolation algorithms.

```
import cv2

from google.colab import files

from google.colab.patches import cv2_imshow

# Upload image

uploaded = files.upload()

filename = list(uploaded.keys())[0]

img = cv2.imread(filename)

nearest = cv2.resize(img, (500,500), interpolation=cv2.INTER_NEAREST)

bilinear = cv2.resize(img, (500,500), interpolation=cv2.INTER_LINEAR)

bicubic = cv2.resize(img, (500,500), interpolation=cv2.INTER_CUBIC)

print("Original")

cv2_imshow(img)

print("Nearest Neighbor")

cv2_imshow(nearest)

print("Bilinear")

cv2_imshow(bilinear)

print("Bicubic")

cv2_imshow(bicubic)
```

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving picture.jfif to picture (2).jfif

Original



Nearest Neighbor:



Bilinear



Bicubic

2) Rotate an Image by Different Angles

Rotates coordinates by specified geometric angles (such as 45°, 90°, and 180°). Requires computing a custom affine rotation matrix and applying coordinate transformations around a specified center point.

```
import cv2

from google.colab import files

from google.colab.patches import cv2_imshow

uploaded = files.upload()

filename = list(uploaded.keys())[0]

img = cv2.imread(filename)

h, w = img.shape[:2]

center = (w//2, h//2)
```

```

for angle in [45, 90, 180]:
    M = cv2.getRotationMatrix2D(center, angle, 1)
    rotated = cv2.warpAffine(img, M, (w,h))
    print(f"Rotation {angle}°")
    cv2_imshow(rotated)

```

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving picture.jfif to picture (3).jfif



Rotation 45°



Rotation 90°



Rotation 180°

3) Crop and Paste a Region of an Image

Extracts a defined rectangular Region of Interest (ROI) from coordinate boundaries and maps/overwrites those pixels onto a distinct target spatial coordinate bounding box within the same canvas.

```

import cv2

from google.colab import files

from google.colab.patches import cv2_imshow

uploaded = files.upload()

filename = list(uploaded.keys())[0]

img = cv2.imread(filename)

roi = img[50:200,50:200]

img[250:400,250:400] = roi

cv2_imshow(img)

```

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving picture.jfif to picture (4).jfif



4) Apply Mean, Gaussian, and Median Filtering

Evaluates linear and non-linear localized smoothing techniques. Demonstrates how box filters average noise, Gaussian kernels blur using weighted normal distributions, and median filters effectively eliminate high-frequency outliers.

```
import cv2

from google.colab import files

from google.colab.patches import cv2_imshow

uploaded = files.upload()

filename = list(uploaded.keys())[0]

img = cv2.imread(filename)

mean = cv2.blur(img, (5,5))

gaussian = cv2.GaussianBlur(img, (5,5), 0)

median = cv2.medianBlur(img,5)

print("Original")

cv2_imshow(img)

print("Mean Filter")

cv2_imshow(mean)

print("Gaussian Filter")
```

```
cv2_imshow(gaussian)

print("Median Filter")

cv2_imshow(median)
```

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving picture.jfif to picture (5).jfif



Original



Mean Filter



Gaussian Filter



Median Filter

5) Add Salt-and-Pepper Noise to an Image

Simulates digital transmission or sensor errors by randomly reassigning a configured subset percentage of pixels to extreme values: either pure black (0) or pure white (255).

```
import cv2

import numpy as np
```

```
from google.colab import files
from google.colab.patches import cv2_imshow
uploaded = files.upload()
filename = list(uploaded.keys())[0]
img = cv2.imread(filename)
noisy = img.copy()
prob = 0.02
for i in range(noisy.shape[0]):
    for j in range(noisy.shape[1]):
        r = np.random.random()
        if r < prob:
            noisy[i,j] = 0
        elif r > 1-prob:
            noisy[i,j] = 255
cv2_imshow(noisy)
```

Upload widget is only available when the cell has been executed in the current browser session.
Please rerun this cell to enable.

Saving picture.jfif to picture (6).jfif



6) Perform Histogram Calculation and Visualization

Quantifies the global frequency distribution of tonal values within an image, tracking exactly how many pixels reside at each discrete brightness level from 0 to 255.

```
import cv2

import matplotlib.pyplot as plt

from google.colab import files

uploaded = files.upload()

filename = list(uploaded.keys())[0]

img = cv2.imread(filename,0)

hist = cv2.calcHist([img],[0],None,[256],[0,256])

plt.figure(figsize=(8,4))

plt.plot(hist,color='black')

plt.title("Histogram")

plt.xlabel("Pixel Intensity")

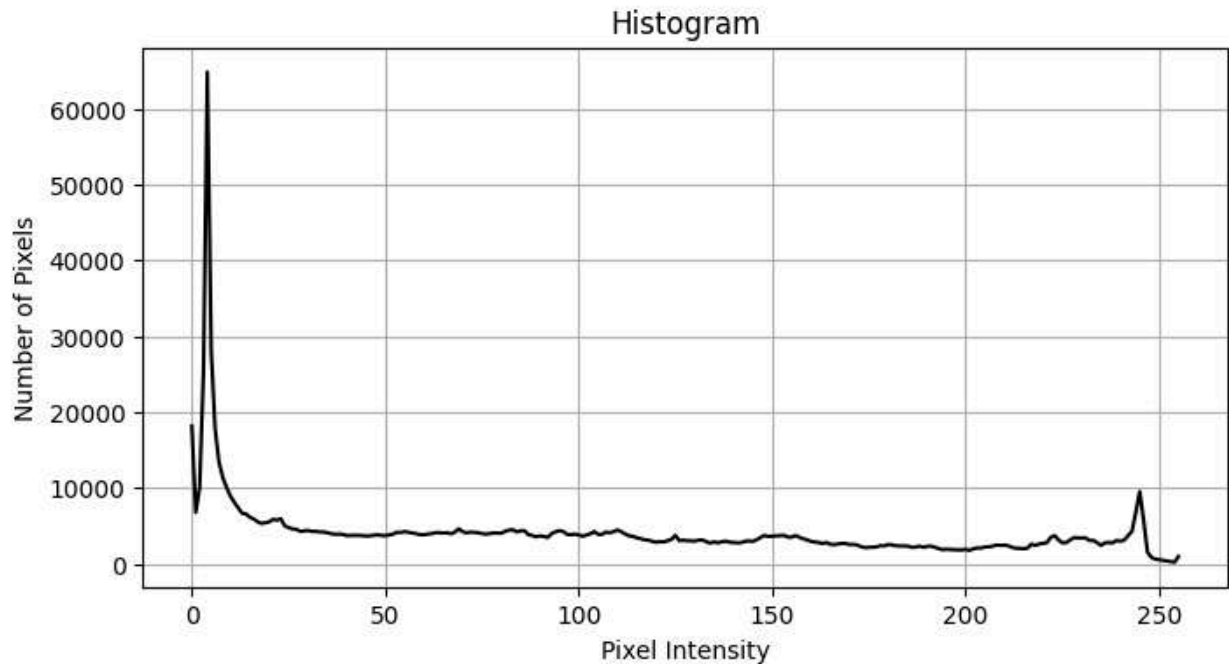
plt.ylabel("Number of Pixels")

plt.grid()

plt.show()
```

Upload widget is only available when the cell has been executed in the current browser session.
Please rerun this cell to enable.

Saving picture.jfif to picture (7).jfif



7) Perform Histogram Equalization

Enhances image contrast globally by stretching the dynamic range of a low-contrast grayscale histogram, mapping its cumulative distribution function (CDF) to a linear uniform distribution.

```
import cv2

from google.colab import files

from google.colab.patches import cv2_imshow

uploaded = files.upload()

filename = list(uploaded.keys())[0]

img = cv2.imread(filename,0)

equalized = cv2.equalizeHist(img)

print("Original")

cv2_imshow(img)

print("Equalized")

cv2_imshow(equalized)
```

Upload widget is only available when the cell has been executed in the current browser session.
Please rerun this cell to enable.

Saving picture.jfif to picture (8).jfif



Original

Equalized

8) Detect Edges Using Sobel Operator

Approximates first-order spatial derivatives by convolving an image with small, directional integer kernels to compute vertical (Gx) and horizontal (Gy) pixel gradients, combining them into a final edge magnitude representation.

```
import cv2

from google.colab import files

from google.colab.patches import cv2_imshow

uploaded = files.upload()

filename = list(uploaded.keys())[0]

img = cv2.imread(filename,0)

sobelx = cv2.Sobel(img,cv2.CV_64F,1,0,ksize=3)

sobely = cv2.Sobel(img,cv2.CV_64F,0,1,ksize=3)

cv2_imshow(cv2.convertScaleAbs(sobelx))

cv2_imshow(cv2.convertScaleAbs(sobely))
```

Upload widget is only available when the cell has been executed in the current browser session.
Please rerun this cell to enable.

Saving picture.jfif to picture (9).jfif



9) Detect Edges Using Canny Edge Detection

An advanced multi-stage edge extractor involving Gaussian noise reduction, intensity gradient calculation, non-maximum suppression to thin edges, and dual-threshold hysteresis to chain continuous boundaries.

```
import cv2

from google.colab import files

from google.colab.patches import cv2_imshow

uploaded = files.upload()

filename = list(uploaded.keys())[0]

img = cv2.imread(filename,0)

edges = cv2.Canny(img,100,200)

cv2_imshow(edges)
```

Upload widget is only available when the cell has been executed in the current browser session.
Please rerun this cell to enable.

Saving picture.jfif to picture (10).jfif

